



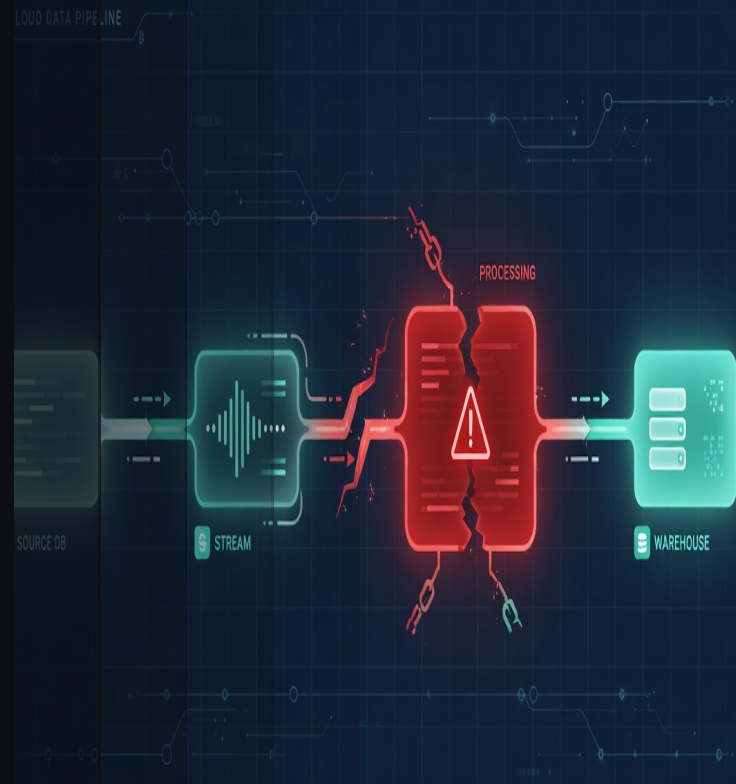
ELASTICSEARCH AGENT BUILDER HACKATHON
2026

PipelineGuard

A tool-driven triage agent for real pipeline failures — built 100% inside Elastic Agent Builder.

Venu Madhav Pentala

Data Engineer · Elastic Dallas User Group
Improving Dallas · Plano, TX · July 21, 2026



Real life looks like this at 2 AM



40+

hours I lost last year just diagnosing pipeline failures — not even fixing them

Log hunting

thousands of lines, one real error

Runbook scavenger hunt

Confluence · Slack · tickets from 2024

Ticket theater

“something is wrong — please investigate”

Tonight: a 30-minute deep dive — live demo, how it's built inside Agent Builder, real tradeoffs, and a pattern you can steal.



PipelineGuard

Fully tool-driven AI triage — 100% inside Elastic Agent Builder. No external orchestration, no custom Python apps, no LangChain.

HOURS → < 60 SEC

manual ritual → agent report

1 DETECT

ES|QL

Time-series-aware queries on real-time logs automatically surface only HIGH-severity errors.

2 RETRIEVE

semantic_text

Vector search over engineering runbooks pulls the exact remediation steps for that error.

3 REPORT

Markdown

A clean triage report: root cause with evidence, step-by-step instructions, and time saved.



The 2 AM report I always wanted

● ● ● PipelineGuard · Triage Report

generated in 47 s

Pipeline Triage Report — 2026-02-26

service: pg-sync env: prod high_errors: 2

ISSUE 1 · ERR-DB-001 — ConnectionTimeout

ROOT CAUSE Postgres exhausted its connection pool

EVIDENCE active=100 max=100 waiters=37 timeout=30 s

FIX 1) restart pgBouncer 2) raise max_connections in Terraform

TIME SAVED ≈ 45 minutes vs manual triage

ISSUE 2 · ERR-DB-002 — PoolStarvation

ROOT CAUSE downstream pool wait exceeded SLA

FIX drain idle sessions · scale pool · re-run pg-sync job

TIME SAVED

45 min vs manual log + runbook
hunt

RECOMMENDED ORDER

- 1 stabilize pool (pgBouncer)
- 2 apply Terraform connection limit
- 3 re-run pg-sync job
- 4 verify lag & error rate

HUMAN STILL DECIDES

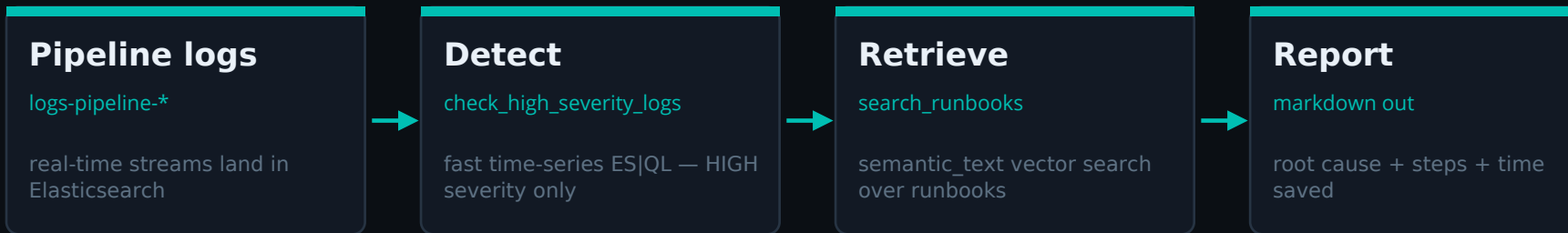
the agent triages — the engineer reviews and executes



How PipelineGuard is wired

Every step native to Elastic Agent Builder — zero external orchestration code

agent reasons: "do I need remediation knowledge?"



BUILT WITH — ALL NATIVE



KEY INSIGHT **do the fast, cheap ES|QL filtering first — only pay for vector search when it actually matters**



How the agent decides



THE RULE **only invoke the more expensive retrieval when the fast detection layer confirms it's worth it**



— THE DIFFERENCE

Same failure. Same runbooks. Different experience.

BEFORE

45-90 min

- 12+ tabs — Kibana, Confluence, Slack, Jira
- 20 minutes of grep and scroll
- runbook hunt through docs that may be stale
- vague ticket: “something is wrong — investigate”

vs

AFTER

< 60 seconds

- one action in Agent Builder
- ES|QL filtered intelligently — then targeted retrieval
- the agent connected the dots — evidence included
- actionable report — you still decide

The agent does the triage. You still get to do the thinking.



Three pieces you can take and apply

01

ES|QL detection

- time-series aware, not keyword grep
- filter aggressively — HIGH severity only
- do the cheap work first — the foundation

02

semantic_text RAG

- runbooks as first-class, well-structured docs
- vector retrieval finds the exact play
- doc quality determines answer quality

03

Tool + prompt design

- force a strict, consistent output schema
- give every tool a clear contract
- design for actionability, not explanation

Not limited to data pipelines — alert triage, infrastructure issues, application errors: anywhere you have logs + documented remediation steps.



What it actually took to make this work

80%

Prompt engineering

structured JSON output, a clear role, good examples in the prompt — moved the needle most

GIGO

Runbook quality

beats model size, every time — clean docs outperform clever prompts on messy ones

0

Extra infrastructure

ES|QL + semantic_text native in Agent Builder — platform velocity is real

HITL

Humans still required

excellent at triage — not reliable enough for autonomous action on prod

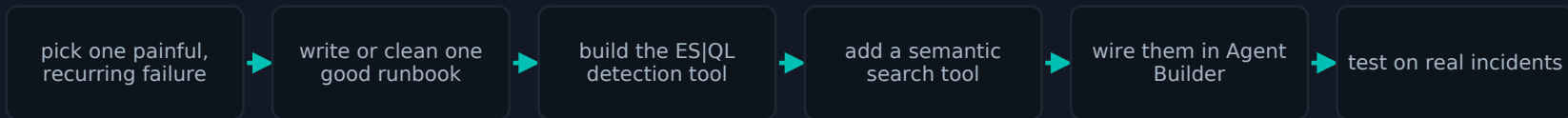
“The agent once suggested restarting the wrong service — the runbook title was ambiguous. That taught me more about documentation hygiene than any prompt engineering ever did.”



What I hope you leave with tonight

- 1 A practical, repeatable pattern — ES|QL log triage + semantic_text RAG inside Agent Builder
- 2 Concrete strategies for structuring tools and prompts so the output is usable and reliable
- 3 A realistic picture of what tool-driven agents do well right now — and where they still need humans

HOW TO GET STARTED — START SMALL



you don't need to build the whole thing at once



Thank you.

The agent does the triage.
You still get to do the thinking.

LOOM · 3 MIN loom.com/share/d20ec2f46ff5479e8d5fa9e1e2cc1765

GITHUB github.com/madhavvan/pipelineguard-elastic-agent-builder

EMAIL pentalavenumadhav@gmail.com



the repo



3-min demo

Questions — I'm especially interested in the failure modes you're fighting right now.